



Chapter 4. ADO.NET

1



Contents

Chapter 4. ADO.NET

- Database
- Structured Query Language
- Case study
- ADO.NET Concepts
- ADO.NET Objects
- DataGridView
- Homework

Slide 2

2



Database

Chapter 4. ADO.NET

- Integrated collection of data (tập hợp của dữ liệu đã hợp thành một thể thống nhất)
- Database management system (DBMS)
 - Provides mechanisms for storing and organizing data in a way that is consistent with database's format
 - Allows storage and access to database without knowledge of internal representation

Slide 3

3



Database

Chapter 4. ADO.NET

- Relational Databases most popular
 - Use Structured Query Language to perform queries (search) and manipulate data
 - Programming languages need an interface to interact with relational databases
- Logical representation of data
 - Relationships can be considered without concern for physical structure of data

Slide 4

4



Database

Microsoft
.net

Chapter 4. ADO.NET

- Composed of tables
 - Rows called records
 - Columns called fields
- Primary key: field that contains unique data
 - Each record can be identified by at least one distinct value

Slide 5

5



Database

Microsoft
.net

Chapter 4. ADO.NET

- Foreign key
 - Field for which every entry has a unique value in another table and where the field in the other table is the primary key for that table
 - Rule of Referential Integrity: every foreign-key field value must appear in another table's primary-key field
 - One to many relationship: A foreign key can appear many times in its own table, but only once as primary key in another table

Slide 6

6

Microsoft
.net



Case study

Chapter 4. ADO.NET

- Student Management System
- Design tables
 - Table KHOADT
 - MAKHOA nvarchar 3
 - TENKHOA nvarchar 50
 - Table NGANH
 - MAKHOA nvarchar 3
 - MANGANH nvarchar 3
 - TENNGANH nvarchar 50

Slide 7

7

Microsoft
.net



Case study

Chapter 4. ADO.NET

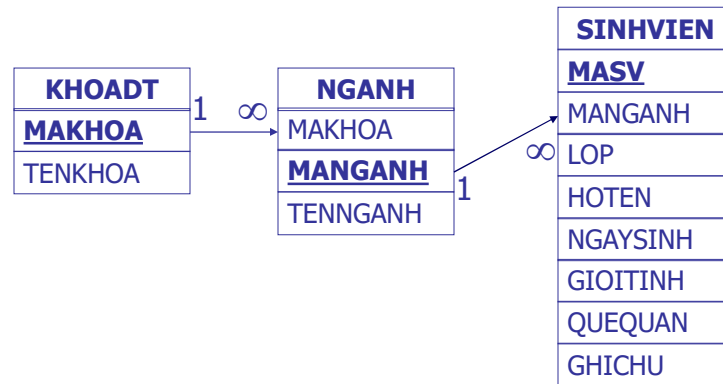
- Design tables
 - Table SINHVIEN
 - MASV nvarchar 7
 - MANGANH nvarchar 3
 - LOP nvarchar 7
 - HOTEN nvarchar 35
 - NGAYSINH nvarchar 10
 - GIOITINH nvarchar 3
 - QUEQUAN nvarchar 50
 - GHICHU nvarchar 50

Slide 8

8

Case study

■ Relational Database



Slide 9

9

Structured Query Language

- Chapter 4. ADO.NET
- Used to request data (perform queries) and manipulate data
 - SQL Statements
 - Select
 - Insert
 - Update
 - Delete

Slide 10

10



Structured Query Language

Chapter 4. ADO.NET

- **SELECT Statement**
 - Extracts information from one or more tables in a database.
 - Basic form: **SELECT * FROM** tableName **WHERE** criteria.
- **INSERT Statement**
 - Inserts a new record into a table
 - Form: **INSERT INTO** tableName(fieldName1) **VALUES** (value1)
 - Values must match field names in order and type

Slide 11

11



Structured Query Language

Chapter 4. ADO.NET

- **UPDATE Statement**
 - Modifies data in a table
 - Form: **UPDATE** tableName **SET** fieldName1 = value1 **WHERE** criteria
- **DELETE Statement**
 - Removes data from a table
 - Form: **DELETE FROM** tableName **WHERE** criteria

Slide 12

12



ADO.NET Concepts

Chapter 4. ADO.NET

- ADO.NET = ActiveX Data Objects
- A set of libraries included within the .NET Framework.
- ADO.NET objects are contained in the **System.Data** namespace.

Slide 13

13



ADO.NET Concepts

Chapter 4. ADO.NET

- All ADO.NET objects can be separated into 2 categories:
 - **Connected:** Objects that communicate directly with the database.
 - **Disconnected:** Objects that allow the user to work with the data offline.

Slide 14

14

Microsoft
.net



ADO.NET Concepts

Chapter 4. ADO.NET

■ **Connected Objects**

- Connection
- Transaction
- DataAdapter
- Command
- Parameter
- DataReader

■ **Disconnected Objects**

- DataSet
- DataTable
- DataView
- DataRow
- DataColumn
- Constraint
- DataRelation

Slide 15

15

Microsoft
.net



ADO Concepts

Chapter 4. ADO.NET

- The ADO.NET Object Model
 - Objects of System.Data
 - .NET data providers
- ADO.NET namespace hierarchy
 - Organizes the object model
 - Includes:
 - System.Data
 - System.Data.OleDb
 - System.Data.Common
 - System.Data.SqlClient
 - System.Data.SqlTypes

Slide 16

16



ADO Concepts

Chapter 4. ADO.NET

- **System.Data:** Consists of the classes that constitute the ADO.NET architecture, which is the primary data access method for managed applications.
- **System.Data.OleDb:** Classes that make up the .NET Framework Data Provider for OLE DB - compatible data sources. These classes allow you to connect to an OLE DB data source, execute commands against the source, and read the results.

Slide 17

17



ADO Concepts

Chapter 4. ADO.NET

- **System.Data.SqlClient:** Classes that make up the .NET Framework Data Provider for SQL Server, which allows you to connect to SQL Server 7.0, execute commands, and read results. The **System.Data.SqlClient** namespace is similar to the **System.Data.OleDb** namespace, but is optimized for access to SQL Server 7.0 and later.

Slide 18

18

Chapter 4. ADO.NET

Microsoft
.net

ADO Concepts

- **System.Data.Common:** Contains classes shared by the .NET Framework data providers. Data providers describe a collection of classes used to access a data source, such as a database, in the managed space.
- **System.Data.SqlTypes:** Provides classes for native data types within SQL Server. These classes provide a safer, faster alternative to other data types.

Slide 19

19

Chapter 4. ADO.NET

Microsoft
.net

ADO.NET-related Namespaces

ADO.NET

System.Data

.OleDb .Common .SqlClient .SqlTypes

Slide 20

20



Introducing the Objects

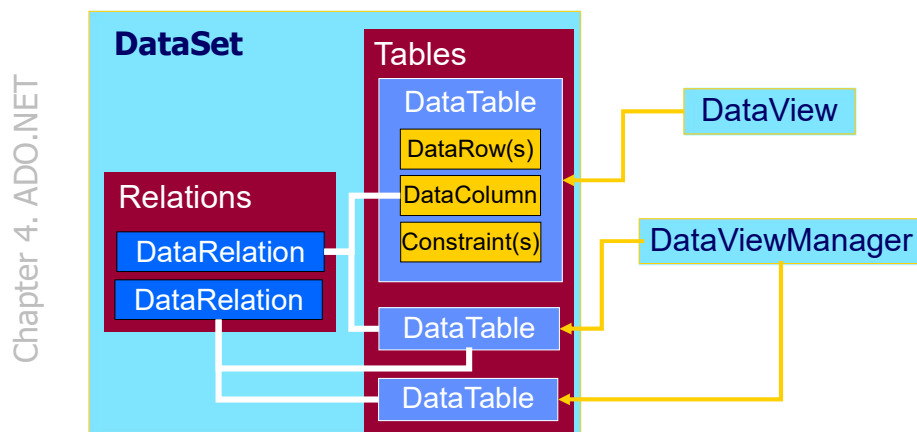
System.Data	Contains the "main" classes of ADO.NET
DataSet	Represents an in-memory cache of data.
DataTable	Represents one table of in-memory data.
DataRow	Represents a row of data in a DataTable .
DataColumn	Represents a column of data in a DataTable .
DataRelation	Used to relate 2 DataTables to each other.
DataViewManager	Used to create views on DataSets

Slide 21

21



DataSet Object



Slide 22

22

Chapter 4. ADO.NET





DataSet Object

- An in-memory cache of data from a data source.
- Common way to represent and manipulate data.
- Designed to be disconnected from the data source.

Slide 23

23

Chapter 4. ADO.NET





DataSet Object

- Methods
 - Collections are used to add, remove tables, relations
- Properties of Interest
 - **Tables:** Returns the collection of **DataTable** objects
 - **Relations:** Returns the collection of **DataRelations**
 - **Namespace:** Gets or sets the namespace of the **DataSet**

Slide 24

24

Chapter 4. ADO.NET

Microsoft
.net

DataTable Object

The diagram illustrates a **DataSet** containing two **DataTable** objects. Each **DataTable** is represented as a grid with 3 rows and 6 columns. The rows are indexed 0, 1, and 2, with arrows pointing to each row. The columns are indexed 0, 1, and 2, with arrows pointing to each column. The label **DataColumn** is placed above the columns, and **DataRow** is placed to the left of the rows.

Slide 25

25

Chapter 4. ADO.NET

Microsoft
.net

DataTable Object

- May be mapped to a physical table in the data source
- Can be related to one another through **DataRelations**
- Common Properties
 - **Columns:** Returns **ColumnsCollection** of **DataColumns**
 - **Rows:** Returns **RowsCollection** of **DataRows**

Slide 26

26

Microsoft
.net



DataTable Object

Chapter 4. ADO.NET

- Create a **DataTable** and add it to a **DataSet**

```

DataSet ds = new DataSet();
// Create DataTable object: "Customers"....
DataTable dt= new DataTable("Customers");
// 1. Explicitly create and Add a DataColumn...
DataColumn dc;
dc = new DataColumn("CustID", Type.GetType("System.Int16"));
dt.Columns.Add(dc);
// 2. Implicitly Create and Add columns (DataColumn)....
dt.Columns.Add("First_Name",Type.GetType("System.String"));
dt.Columns.Add("Last_Name", Type.GetType("System.String"));
// Add the DataTable object to the DataSet
ds.Tables.Add(dt);

```

Slide 27

27

Microsoft
.net



ADO.NET Data Providers

Chapter 4. ADO.NET

- A collection of classes for accessing data sources
 - Microsoft SQL Server™ 2000, SQL Server 7
 - Any OLE Database (OLE DB) providers
 - Including: Oracle, JET, and SQL OLE DB Providers
- Establish connection between **DataSets** and data stores.

Slide 28

28

Chapter 4. ADO.NET

Microsoft
.net

ADO.NET Data Providers

- Two .NET data providers
 - OLE Database: via the **System.Data.OleDb** namespace
 - SQL Server: via the **System.Data.SqlClient** namespace
- System.Data.OleDb** and **System.Data.SqlClient** are *the* .NET data provider.

Slide 29

29

Chapter 4. ADO.NET

Microsoft
.net

.NET Data Providers Hierarchy

The diagram illustrates the hierarchy of .NET data providers. It features a yellow pyramid. At the top of the pyramid is a blue box labeled **.Common** with the text "Contains classes shared by both". An arrow points from this box to the center of the pyramid. In the center, there is a dark red box labeled **System.Data**. Below this, the pyramid is divided into two columns. The left column is labeled **.SqlClient** and lists the classes: **SqlCommand**, **SqlConnection**, **SqlDataReader**, and **SqlDataAdapter**. The right column is labeled **.OleDb** and lists the classes: **OleDbCommand**, **OleDbConnection**, **OleDbDataReader**, and **OleDbDataAdapter**.

Slide 30

30



Connection

- Represent a unique session with a data source
- Create, open, close a connection to a data source
- Functionality and methods to perform transactions

Slide 31

31



Connection

- OleDbConnection & SqlConnection Example

```
string conStr="Provider=Microsoft.Jet.OLEDB.4.0;" +
    "Data Source=<DataName>";
OleDbConnection myConn = new OleDbConnection(conStr);
myConn.Open();
```

```
string conStr="Data Source = <Computer Name>;" +
    "Persist Security Info = true;" +
    "Initial Catalog = <DataName>;" +
    "User Id =name; Password=psw;" +
    "Connect Timeout = <seconds>";
SqlConnection myConn = new SqlConnection(conStr);
myConn.Open();
```

Slide 32

32

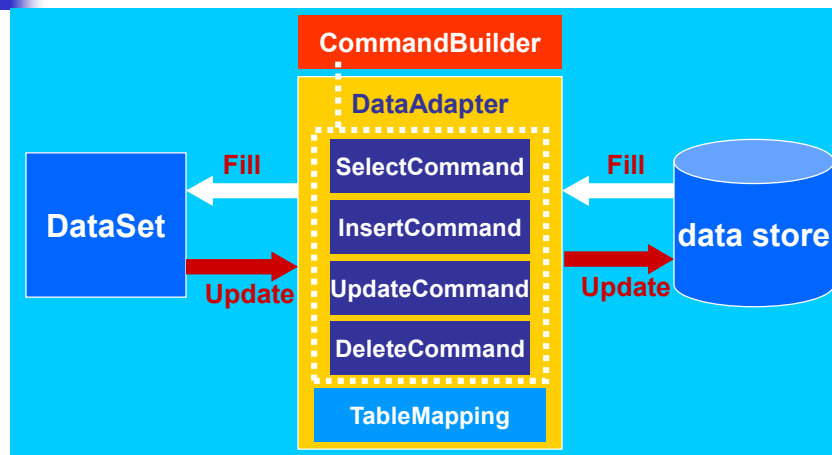
SqlDataAdapter Class

- Represents a set of data commands and a database connection that are used to fill the DataSet and update a database.
- OleDbDataAdapter and SqlDataAdapter are similar in data manipulation data.
- Bridge between the **DataSet** and the data store.

Slide 33

33

SqlDataAdapter Class



Slide 34

34



SqlDataAdapter Class

Chapter 4. ADO.NET

- Properties of Interest
 - **SelectCommand**: Gets or sets select command used to select records in the data source.
 - **DeleteCommand**: Gets or sets delete command used to delete records from the data set.
 - **InsertCommand**: Gets or sets insert command used to insert new records into the data source.
 - **UpdateCommand**: Gets or sets update command used to update records in the data source.

Slide 35

35



SqlDataAdapter Class

Chapter 4. ADO.NET

- Properties of Interest
 - **TableMappings**: Maps source table and a **DataTable**
- **SqlCommand**s retrieved or set by command properties
- Methods of the **DataAdapter** class
 - `Fill(DataSet dataSet);`
 - `Update(DataSet dataSet);`

Slide 36

36

Microsoft
.net



SqlDataAdapter Class

Chapter 4. ADO.NET

■ **Fill** method Example

```
string conStr = "Data Source = may01;" +
    "Initial Catalog = QLSV;" +
    "Persist Security Info = true;" +
    "User Id =sa; Password=sa; Connect Timeout =50 ";
string sqlStr = "SELECT * FROM KHOADT";
SqlDataAdapter da = new SqlDataAdapter(sqlStr, conStr);
DataSet myDataSet = new DataSet();
da.Fill(myDataSet, "KHOADT");
DataTable myTable = myDataSet.Tables["KHOADT"];
//Continue doing something...
```

Slide 37

37

Microsoft
.net



SqlDataAdapter Class

Chapter 4. ADO.NET

■ **Pattern for Connection Database**

```
1.) Declare connection
try {
    1.) Request connection to database
    2.) Execute SQL commands
    3.) Process result
    4.) Release Resources
} catch (Exception){
    Handle exception
} finally {
    try {
        5.) Close connection
    } catch (Exception)
    { Handle exception }
}
```

Slide 38

38

Chapter 4. ADO.NET

Microsoft .net

SqlDataAdapter Class

- Example for Connection Database

```
string conStr = "Data Source = may01;Initial Catalog =QLSV;..."
string sqlStr = "SELECT * FROM KHOADT";
SqlDataAdapter da = new SqlDataAdapter(sqlStr, conStr);
DataSet myDataSet = new DataSet();
try
{
    da.Fill(myDataSet, "KHOADT");
}
catch (SqlException ex)
{
    MessageBox.Show(ex.Message, "Error " + ex.Number.ToString());
    return;
}
//Continue doing something...
```

Slide 39

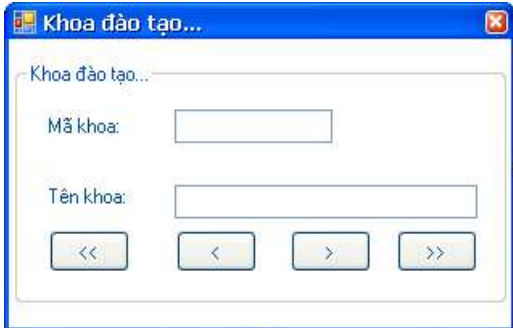
39

Chapter 4. ADO.NET

Microsoft .net

SqlDataAdapter Class

- Example: Display records in table KHOADT



Name: txtMakhoa
Name: txtTen

Name: btnFirst, btnPrevious, btnNext, btnLast

Slide 40

40



SqlDataAdapter Class

- Open namespace
 - `using System.Data.SqlClient;`
- Declare in class

```
private string constr = "Data Source = may01;" +
    "Initial Catalog = QLSV;" +
    "persist security info = true;" +
    "User Id=sa; Password=sa; Connect Timeout =50";
private SqlDataAdapter myDataAdapter;
private DataSet myDataSet;
private DataTable myTable;
int pos = 0; //position of current row
```

Slide 41

41



SqlDataAdapter Class

- Load, First, Previous, Next and Last Event

```
private void xemkhoa_Load(object sender, EventArgs e)
{
    string SqlStr = "SELECT * FROM KHOADT";
    myDataAdapter = new SqlDataAdapter(SqlStr, constr);
    myDataSet = new DataSet();
    //Can use try ...catch ...
    myDataAdapter.Fill(myDataSet, "KHOADT");
    myTable = myDataSet.Tables["KHOADT"];
    btnFirst.PerformClick();
}
```

Slide 42

42

Chapter 4. ADO.NET

Microsoft
.net

SqlDataAdapter Class

```
private void btnFirst_Click(object sender, EventArgs e)
{
    if (myTable.Rows.Count == 0) return;
    pos=0;
    txtMakhoa.Text = myTable.Rows[pos]["MAKHOA"].ToString();
    txtTen.Text = myTable.Rows[pos]["TENKHOA"].ToString();
}

private void btnPrevious_Click(object sender, EventArgs e)
{
    if (myTable.Rows.Count == 0) return;
    pos--;
    if (pos < 0) pos = 0;
    txtMakhoa.Text = myTable.Rows[pos]["MAKHOA"].ToString();
    txtTen.Text = myTable.Rows[pos]["TENKHOA"].ToString();
}
```

Slide 43

43

Chapter 4. ADO.NET

Microsoft
.net

SqlDataAdapter Class

```
private void btnNext_Click(object sender, EventArgs e)
{
    if (myTable.Rows.Count == 0) return;
    pos++;
    if (pos > myTable.Rows.Count - 1) pos = myTable.Rows.Count - 1;
    txtMakhoa.Text = myTable.Rows[pos]["MAKHOA"].ToString();
    txtTen.Text = myTable.Rows[pos]["TENKHOA"].ToString();
}

private void btnLast_Click(object sender, EventArgs e)
{
    if (myTable.Rows.Count == 0) return;
    pos = myTable.Rows.Count - 1;
    txtMakhoa.Text = myTable.Rows[pos]["MAKHOA"].ToString();
    txtTen.Text = myTable.Rows[pos]["TENKHOA"].ToString();
}
```

Slide 44

44



SqlCommand Class

Chapter 4. ADO.NET

- Represents a query to execute on the data source
 - May be a SQL statement or stored procedure
- Properties of Interest
 - **Connection**: Get or set the data source connection.
 - **CommandText**: Get or set the query (text) command.

Slide 45

45



SqlCommand Class

Chapter 4. ADO.NET

- Properties of Interest
 - **CommandType**: Get/set how the command is interpreted.
 - **Text**, **StoredProcedure**, or **TableDirect**
 - **CommandTimeout**: The seconds until connection timeout.
- Methods of Interest
 - **ExecuteNonQuery** to execute SQL statements on the data source

Slide 46

46

SqlCommand Class

- Code pattern
 - Declare variables
 - `private string conStr = "Data Source = ...;";`
 - `private SqlConnection myConnection;`
 - `private SqlCommand myCommand;`
 - Open connection
 - `myConnection = new SqlConnection(conStr);`
 - `myConnection.Open();`
 - Execute SQL statements
 - `myCommand = new SqlCommand(sqlStr, myConnection);`
 - `myCommand.ExecuteNonQuery();`

Slide 47

47

SqlCommand Class

■ Example

Name property

txtMakhoa

txtTen

btnNew, btnEdit, btnDelete

btnSave, btnCancel, btnClose



Slide 48

48

Microsoft
.net



SqlCommand Class

Chapter 4. ADO.NET

- Declare variables in class

```
private string conStr = "Data Source = may01;..."
// Declare to Insert, Delete, Update data
private SqlConnection myConnection;
private SqlCommand myCommand;
// Declare to Load data to DataTable
private SqlDataAdapter myDataAdapter;
private DataSet myDataSet;
private DataTable myTable;
private bool modeNew;
private string oldMakhoa;
private int pos = 0;
```

Slide 49

49

Microsoft
.net



SqlCommand Class

Chapter 4. ADO.NET

- Adds the function to enable or disable textboxes and buttons

```
private void SetControls(bool edit)
{
    txtMakhoa.Enabled = edit;
    txtTen.Enabled = edit;
    btnNew.Enabled = !edit;
    btnEdit.Enabled = !edit;
    btnDelete.Enabled = !edit;
    btnSave.Enabled = edit;
    btnCancel.Enabled = edit;
}
```

Slide 50

50

Microsoft
.net



SqlCommand Class

- Adds the function to load table to DataTable

```
private void LoadTable()
{
    string sSql = "SELECT * FROM KHOADT";
    myDataAdapter = new SqlDataAdapter(sSql, conStr);
    myDataSet = new DataSet();
    myDataAdapter.Fill(myDataSet, "KHOADT");
    myTable = myDataSet.Tables["KHOADT"];
}
```

- Can use try ...catch...finally...to connect data

Slide 51

51

Microsoft
.net



SqlCommand Class

- Load Event

```
private void Nhaphkhoa_Load(object sender, EventArgs e)
{
    LoadTable();
    btnFirst.PerformClick();
    SetControls(false);
    //used to execute SqlCommand
    myConnection = new SqlConnection(conStr);
    myConnection.Open();
}
```

Slide 52

52

Microsoft
.net



SqlCommand Class

Chapter 4. ADO.NET

- **btnNew** and **btnEdit** Click Event

```
private void btnNew_Click(object sender, EventArgs e)
{
    modeNew = true; // btnNew is clicked
    SetControls(true);
    txtMakhoa.Text = "";
    txtTen.Text = "";
    txtMakhoa.Focus();
}

private void btnEdit_Click(object sender, EventArgs e)
{
    oldMakhoa = txtMakhoa.Text;
    modeNew = false; // btnEdit is clicked
    SetControls(true);
    txtMakhoa.Focus();
}
```

Slide 53

53

Microsoft
.net



SqlCommand Class

Chapter 4. ADO.NET

- **btnDelete** Click Event

```
private void btnDelete_Click(object sender, EventArgs e)
{
    if (myTable.Rows.Count == 0 ) return;
    string sqlStr = "DELETE FROM KHOADT WHERE MAKHOA =" +
        txtMakhoa.Text + "'";
    myCommand = new SqlCommand(sqlStr, myConnection);
    myCommand.ExecuteNonQuery();
    txtMakhoa.Text = "";
    txtTen.Text = "";
}
//Exercise: Add try ...catch ...finally to this event.
```

Slide 54

54

Microsoft
.net



SqlCommand Class

■ btnSave Click Event

```
private void btnSave_Click(object sender, EventArgs e)
{
    string sqlStr;
    if (modeNew)
    {
        sqlStr = "INSERT INTO KHOADT (MAKHOA,TENKHOA)" +
            "VALUES ('" + txtMakhhoa.Text + "','" +
            txtTen.Text + "')";
    }
    else
    {
        sqlStr = "UPDATE KHOADT SET MAKHOA = '" + txtMakhhoa.Text +
            "',TENKHOA ='" + txtTen.Text + "'" +
            " WHERE MAKHOA ='" + oldMakhhoa + "'";
    }
    myCommand = new SqlCommand(sqlStr, myConnection);
    // continue next page...
}
```

Slide 55

55

Microsoft
.net



SqlCommand Class

■ btnSave Click Event

```
try
{
    myCommand.ExecuteNonQuery();
    LoadTable();
}
catch (Exception ex)
{
    MessageBox.Show(ex.Message);
    btnFirst.PerformClick();
}
finally
{
    SetControls(false);
}
}
```

Slide 56

56

Chapter 4. ADO.NET

Microsoft .net

SqlCommand Class

- btnCancel Click Event

```
private void btnCancel_Click(object sender, EventArgs e)
{
    SetControls(false);
}
```

- btnClose Click Event

```
private void btnClose_Click(object sender, EventArgs e)
{
    this.Close();
}
```

Slide 57

57

Chapter 4. ADO.NET

Microsoft .net

SqlCommandBuilder Class

- Auto creates Update, Insert, Delete Commands within a SqlDataAdapter

The diagram shows the following components and interactions:

- DataSet** (blue box) on the left.
- DataAdapter** (yellow box) in the center, containing:
 - SelectCommand** (blue box)
 - InsertCommand** (blue box)
 - UpdateCommand** (blue box)
 - DeleteCommand** (blue box)
 - TableMapping** (blue box)
- SqlCommandBuilder** (red box) at the top, connected to the DataAdapter.
- data store** (blue cylinder) on the right.

Interactions:

- A white arrow labeled **Fill** points from the **data store** to the **DataAdapter**.
- A red arrow labeled **Update** points from the **DataAdapter** to the **DataSet**.
- A red arrow labeled **Update** points from the **DataAdapter** to the **data store**.

Slide 58

58

Chapter 4. ADO.NET

Microsoft
.net

SqlCommandBuilder Class

- Basic code pattern
 - Declare variables in the class
 - `private string conStr = "Data Source = ...;";`
 - `private SqlDataAdapter myDataAdapter;`
 - `private SqlCommandBuilder myCommandBuilder;`
 - `private DataSet myDataSet;`
 - `private DataTable myTable;`
 - `private string sqlStr;`
 - Make a SELECT statement
 - `sqlStr = "SELECT .... FROM ....";`

Slide 59

59

Chapter 4. ADO.NET

Microsoft
.net

SqlCommandBuilder Class

- Basic code pattern – usually set in Load Event
 - Makes a SqlDataAdapter
 - `myDataAdapter = new SqlDataAdapter(sqlStr, conStr);`
 - Makes a SqlCommandBuilder
 - `myCommandBuilder = new SqlCommandBuilder(myDataAdapter);`
 - Makes a DataTable
 - `myDataSet = new DataSet();`
 - `myDataAdapter.Fill(myDataSet, "....");`
 - `myTable = myDataSet.Tables["...."];`

Slide 60

60



SqlCommandBuilder Class

Chapter 4. ADO.NET

- Basic code pattern
 - Delete a row
 - `myTable.Rows[pos].Delete();`
 - `myDataAdapter.Update(myTable);`
 - **Note:** pos is row of deleting
 - Add a row
 - `DataRow newRow = myTable.NewRow();`
 - `newRow["MAKHOA"] = txtMakhoa.Text;`
 - `newRow["TENKHOA"] = txtTen.Text;`
 - `myTable.Rows.Add(newRow);`
 - `myDataAdapter.Update(myTable);`

Slide 61

61



SqlCommandBuilder Class

Chapter 4. ADO.NET

- Basic code pattern
 - Edit a row
 - `DataRow editRow = myTable.Rows[pos];`
 - `editRow["MAKHOA"] = txtMakhoa.Text;`
 - `editRow["TENKHOA"] = txtTenkhoa.Text;`
 - `myDataAdapter.Update(myTable);`
 - **Note:** pos is row of editing
 - Reject a row
 - `myTable.RejectChanges();`

Slide 62

62





DataGridView

Chapter 4. ADO.NET

- Used to display data in a customizable grid
- Add columns to DataGridView
 - Clicks right mouse button
 - Chooses **Add column...Or Edit columns...**
 - Chooses **Add...**
 - Important properties of the column
 - **Name:** Indicates the name is used in code
 - **Header text:** The caption text on the column's header cell
 - **DataPropertyName:** The name of the data source property or database column.

Slide 63

63





DataGridView

Chapter 4. ADO.NET

- Common Property
 - **AutoGenerateColumns:** indicates whether columns are created automatically when the DataSource or DataMember properties are set.
 - `dataGridView1.AutoGenerateColumns = false;`
 - `dataGridView1.DataSource = myTable;`
 - **AllowUserToAddRows:** Indicates whether the option to add rows is displayed to the user
 - **AllowUserToDeleteRows:** indicates whether the user is allowed to delete rows

Slide 64

64

Microsoft
.net



DataGridView

Chapter 4. ADO.NET

- Common Event
 - **RowEnter**: Occurs when a row receives input focus and becomes the current row.
 - **e.RowIndex**: returns current row.
 - **e.ColumnIndex**: returns current column.

Slide 65

65

Microsoft
.net



DataGridView

Chapter 4. ADO.NET

- Displays DataTable in DataGridView



Slide 66

66



DataGridView

- Design dataGridView1
- Declare variables in class

```
private string conStr = "Data Source = may01;" +
    "Initial Catalog = QLSV;" +
    "Persist Security Info = true;" +
    "User Id =sa; Password=sa;" +
    "Connect timeout =50";
// Declare variables to load data to DataTable
private SqlDataAdapter myDataAdapter;
private DataSet myDataSet;
private DataTable myTable;
private string sqlStr;
```

Slide 67

67



DataGridView

- Load Event

```
private void Xemkhoa_Load(object sender, EventArgs e)
{
    sqlStr = "SELECT * FROM KHOADT";
    myDataAdapter = new SqlDataAdapter(sqlStr, conStr);
    myDataSet = new DataSet();
    myDataAdapter.Fill(myDataSet, "KHOADT");
    myTable = myDataSet.Tables["KHOADT"];
    //Load data on DataGridView
    dataGridView1.AutoGenerateColumns = false;
    dataGridView1.DataSource = myTable;
}
```

Slide 68

68

Example

- Design the following form

The screenshot shows a Windows application window titled "Nhập, sửa, xóa Khoa đào tạo...". Inside the window, there are two text input fields labeled "Mã khoa:" and "Tên khoa:". Below these fields are six buttons arranged in two rows: "New", "Edit", "Delete" in the top row, and "Save", "Cancel", "Close" in the bottom row. To the right of the buttons is a table with two columns, "Mã khoa" and "Tên khoa", and a scrollable area below it.

Slide 69

69

Example

- Declare variables in class

```
private string conStr = "Data Source = may01;" +
    "Initial Catalog = QLSV;" +
    "Persist Security Info = true;" +
    "User Id =sa; Password=sa;" +
    "Connect Timeout =50";

private SqlDataAdapter myDataAdapter;
private SqlCommandBuilder myCommandBuilder;
// declare variables to load data to DataTable
private DataSet myDataSet;
private DataTable myTable;
private bool modeNew;
private int pos;
```

Slide 70

70



Example

- Adds the method to enable or disable textboxes and buttons

```
private void SetControls(bool edit)
{
    txtMakhoa.Enabled = edit;
    txtTen.Enabled = edit;
    btnNew.Enabled = !edit;
    btnEdit.Enabled = !edit;
    btnDelete.Enabled = !edit;
    btnSave.Enabled = edit;
    btnCancel.Enabled = edit;
}
```

Slide 71

Chapter 4. ADO.NET

71



Example

- Adds the method used for SqlServer Error

```
void DisplayError(SqlException ex)
{
    string ErrorMessage;
    switch (ex.Number)
    {
        case 17:
            ErrorMessage = "Server does not exist or access denied !";
            break;
        case 4060:
            ErrorMessage = "Invalid Database !";
            break;
        case 18456:
            ErrorMessage = "Login Failed !";
            break;
    }
}
```

Slide 72

Chapter 4. ADO.NET

72



Example

```

case 547:
    ErrorMessage = "ForeignKey Violation !";
    break;
case 2627:
case 2601:
    ErrorMessage = "Unique Index/Constraint Violation";
    break;
case 8152:
    ErrorMessage = "String or binary is too long";
    break;
default:
    ErrorMessage = ex.Message;
    break;
}
MessageBox.Show(ErrorMessage, "Error " + ex.Number.ToString());
}

```

Slide 73

73



Example

■ Load Event

```

private void Khoa_Load(object sender, EventArgs e)
{
    string sqlStr = "SELECT * FROM KHOADT";
    myDataAdapter = new SqlDataAdapter(sqlStr, conStr);
    myCommandBuilder = new SqlCommandBuilder(myDataAdapter);
    myDataSet = new DataSet();
    myDataAdapter.Fill(myDataSet, "KHOADT");
    myTable = myDataSet.Tables["KHOADT"];
    //Load data on DataGridView
    dataGridView1.AutoGenerateColumns = false;
    dataGridView1.DataSource = myTable;
    SetControls(false);
}

```

Slide 74

74

Example

■ RowEnter click Event

```
private void dataGridView1_RowEnter(object sender,
                                   DataGridViewCellEventArgs e)
{
    if (dataGridView1.RowCount >= 0)
    {
        pos = e.RowIndex;
        txtMakhoa.Text = dataGridView1.
            Rows[pos].Cells["MAKHOA"].Value.ToString();
        txtTen.Text = dataGridView1.
            Rows[pos].Cells["TENKHOA"].Value.ToString();
    }
}
```

Chapter 4. ADO.NET

Slide 75

75

Example

■ btnNew and btnEdit Click Event

```
private void btnNew_Click(object sender, EventArgs e)
{
    modeNew = true; // btnNew is clicked
    SetControls(true);
    txtMakhoa.Text = "";
    txtTen.Text = "";
    txtMakhoa.Focus();
}
```

```
private void btnEdit_Click(object sender, EventArgs e)
{
    oldMakhoa = txtMakhoa.Text.Trim();
    modeNew = false; // btnEdit is clicked
    SetControls(true);
    txtMakhoa.Focus();
}
```

Chapter 4. ADO.NET

Slide 76

76



Example

■ btnDelete Click Event

```
private void btnDelete_Click(object sender, EventArgs e)
{
    if (myTable.Rows.Count == 0) return;
    if (txtMakhoa.Text.Trim() == "") return;
    myTable.Rows[pos].Delete();
    myDataAdapter.Update(myTable);
}
```

■ btnCancel Click Event

```
private void btnCancel_Click(object sender, EventArgs e)
{
    SetControls(false);
}
```

Slide 77

77



Example

■ btnSave Click Event

```
private void btnSave_Click(object sender, EventArgs e)
{
    if (modeNew)
    {
        DataRow newRow = myTable.NewRow();
        newRow["MAKHOA"] = txtMakhoa.Text;
        newRow["TENKHOA"] = txtTen.Text;
        myTable.Rows.Add(newRow);
    }
    else
    {
        DataRow editRow = myTable.Rows[pos];
        editRow["MAKHOA"] = txtMakhoa.Text;
        editRow["TENKHOA"] = txtTen.Text;
    }
}
```

Slide 78

78

Chapter 4. ADO.NET



Example



```
try
{
    myDataAdapter.Update(myTable);
}
catch (SqlException ex)
{
    myTable.RejectChanges();
    DisplayError(ex);
}
catch (Exception ex)
{
    myTable.RejectChanges();
    MessageBox.Show(ex.Message);
}
finally
{
    SetControls(false);
}
}
```

Slide 79

79

Chapter 4. ADO.NET



ListBox and ComboBox



- **ListBoxes:** Allow users to view and select from items on a list
- **ComboBox:** Combine **TextBox** and drop-down list

Slide 80

80



ListBox and ComboBox

Chapter 4. ADO.NET

- Common Properties
 - **DataSource**: Indicates the list that this control will use to get its items
 - **DisplayMember**: Indicates the property to display for the items in this control
 - **ValueMember**: Indicates the property to use as actual value for the items in this control.
 - **SelectedIndex**: Indicates the currently selected item.
 - **SelectedValue**: the actual value for the currently selected item

Slide 81

81



ListBox and ComboBox

Chapter 4. ADO.NET

- Code pattern
 - Displays KHOADT table in listbox

```

if (myTable.Rows.Count > 0)
{
    comboBox1.DataSource = myTable;
    comboBox1.DisplayMember = "TENKHOA";
    comboBox1.ValueMember = "MAKHOA";
    comboBox1.SelectedIndex = 0;
}
      
```

- Return value
 - comboBox1.SelectedValue

Slide 82

82

Homework

■ Design the form

Slide 83

83

Designs a database class

- Provides a class to manipulate to data source
 - Open a Visual Studio project
 - Add an empty class definition to your project
 - Click on **Project -> Add Windows Form** Item.
 - Select **Class** and rename the file (ex: DataSource.cs).
 - Click **Add** to add the class to your project.
 - Once the report is added, you will see the class design surface.

Slide 84

84

Chapter 4. ADO.NET

Microsoft
.net

Designs a database class

- Declare in class

```
static private string conStr;
static private SqlConnection myConnection;
private SqlDataAdapter myDataAdapter;
```

- Constructors in class

```
public DataSource()
{
}
public DataSource(string computer, string dataSource,
                  string user, string psw)
{
    conStr = "Data Source=..";
}
```

Slide 85

85

Chapter 4. ADO.NET

Microsoft
.net

Designs a database class

- Adds the method used for SqlServer Error

```
void DisplayError(SqlException ex)
{
    string ErrorMessage;
    switch (ex.Number)
    {
        case 17:
            ErrorMessage = "Server does not exist or access denied !";
            break;
        case 4060:
            ErrorMessage = "Invalid Database !";
            break;
        case 18456:
            ErrorMessage = "Login Failed !";
            break;
    }
}
```

Slide 86

86

Chapter 4. ADO.NET

Microsoft
.net

Designs a database class

```

case 547:
    ErrorMessage = "ForeignKey Violation !";
    break;
case 2627:
case 2601:
    ErrorMessage = "Unique Index/Constriant Violation";
    break;
case 8152:
    ErrorMessage = "String or binary is too long";
    break;
default:
    ErrorMessage = ex.Message;
    break;
}
MessageBox.Show(ErrorMessage, "Error " + ex.Number.ToString());
}

```

Slide 87

87

Chapter 4. ADO.NET

Microsoft
.net

Designs a database class

- The function opens a connection

```

public bool IsConnected()
{
    myConnection = new SqlConnection(conStr);
    try
    {
        myConnection.Open();
        return true;
    }
    catch (SqlException ex)
    {
        DisplayError(ex);
        return false;
    }
}

```

Slide 88

88

Microsoft
.net



Designs a database class

- The function fills a DataSet and can update data source from a DataSet, DataTable

```

public DataSet SelectUpdatesqlData(string sqlStr,
                                   string tableName)
{
    myDataAdapter = new SqlDataAdapter(sqlStr, myConnection);
    SqlCommandBuilder myCommandBuilder;
    myCommandBuilder = new SqlCommandBuilder(myDataAdapter);
    DataSet myDataSet = new DataSet();
    myDataAdapter.Fill(myDataSet, tableName);
    return myDataSet;
}

```

Slide 89

89

Microsoft
.net



Designs a database class

- The function updates a DataTable to data source

```

public void UpdatesqlData(DataTable myTable)
{
    try
    {
        myDataAdapter.Update(myTable);
    }
    catch (SqlException ex)
    {
        DisplayError(ex);
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
}

```

Slide 90

90

Microsoft
.net



Designs a database class

Chapter 4. ADO.NET

- The function fills a DataSet but doesn't allow update data source

```
public DataSet SelectUpdatesSqlData(string sqlStr,
                                   string tableName)
{
    myDataAdapter = new SqlDataAdapter(sqlStr, myConnection);
    DataSet myDataSet = new DataSet();
    myDataAdapter.Fill(myDataSet, tableName);
    return myDataSet;
}
```

Slide 91

91

Microsoft
.net



Designs a database class

Chapter 4. ADO.NET

- The function updates sql string

```
public void UpdatesSqlData(string UpdateString)
{
    SqlCommand myCommand;
    myCommand = new SqlCommand(UpdateString, myConnection);
    try
    {
        myCommand.ExecuteNonQuery();
    }
    catch (SqlException ex)
    {
        DisplayError(ex);
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
}
```

Slide 92

92

Chapter 4. ADO.NET

Microsoft .net

Designs a database class

- The function inserts sql string

```
public void InsertSqlData(string InsertString)
{
    SqlCommand myCommand;
    myCommand = new SqlCommand(InsertString, myConnection);
    try
    {
        myCommand.ExecuteNonQuery();
    }
    catch (SqlException ex)
    {
        DisplayError(ex);
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
}
```

Slide 93

93

Chapter 4. ADO.NET

Microsoft .net

Designs a database class

- The function deletes sql string

```
public void DeleteSqlData(string DeleteString)
{
    SqlCommand myCommand;
    myCommand = new SqlCommand(DeleteString, myConnection);
    try
    {
        myCommand.ExecuteNonQuery();
    }
    catch (SqlException ex)
    {
        DisplayError(ex);
    }
    catch (Exception ex)
    {
        MessageBox.Show(ex.Message);
    }
}
```

Slide 94

94

Microsoft
.net



SqlDataReader

Chapter 4. ADO.NET

- Forward-only data access
- “Lightweight” programming model
 - Less overhead than using **OleDbDataAdapter**
- Instantiated & returned by **SqlCommand.ExecuteReader**
- Ties up the **SqlCommand** until it is finished reading

Slide 95

95

Microsoft
.net



SqlDataReader

Chapter 4. ADO.NET

- Properties of Interest
 - **FieldCount**: Returns the number of fields in the result set
 - **RecordsAffected**: Number of affected records
- Methods to retrieve data
 - By column type and/or index: **GetValue**; **GetString**; etc.
 - **Read()**: Advances reader to next record
 - **NextResult()**: Advanced to next result set in batch
 - **GetValues()**: Gets the current row

Slide 96

96

Microsoft
.net



SqlDataReader Sample

Chapter 4. ADO.NET

- Code pattern to add Items to checkBox

```

//String for creating the SqlConnection "myConn" not shown
SqlConnection myConn = new SqlConnection(conStr);
myConn.Open();
string sSql = "SELECT * FROM NGANH";
SqlCommand myCmd = new SqlCommand(sSql, myConn);
SqlDataReader myReader = myCmd.ExecuteReader();
while (myReader.Read())
{
    string str = myReader.GetValue(1).ToString();
    checkedListBox1.Items.Add(str);
}
myReader.Close();
myConn.Close();

```

Slide 97

97

Microsoft
.net



DataRelation Object

Chapter 4. ADO.NET

- Used to create logical relations between your data
 - Create relations between two (2) **DataTable** objects
 - Requires a **DataColumn** object from each **DataTable**
 - The **DataType** of both **DataColumns** must be the same
 - Cannot relate a **Int32 DataColumn** and a **String DataColumn**
 - The relation is named (by you!)

Slide 98

98



DataRelation Object

Chapter 4. ADO.NET

- Makes relational navigation possible
- **RelationsCollection** used to hold/group them
 - Accessed through the **DataSet's Relations** property

Slide 99

99



DataRelation Object

Chapter 4. ADO.NET

- DataRelation Example

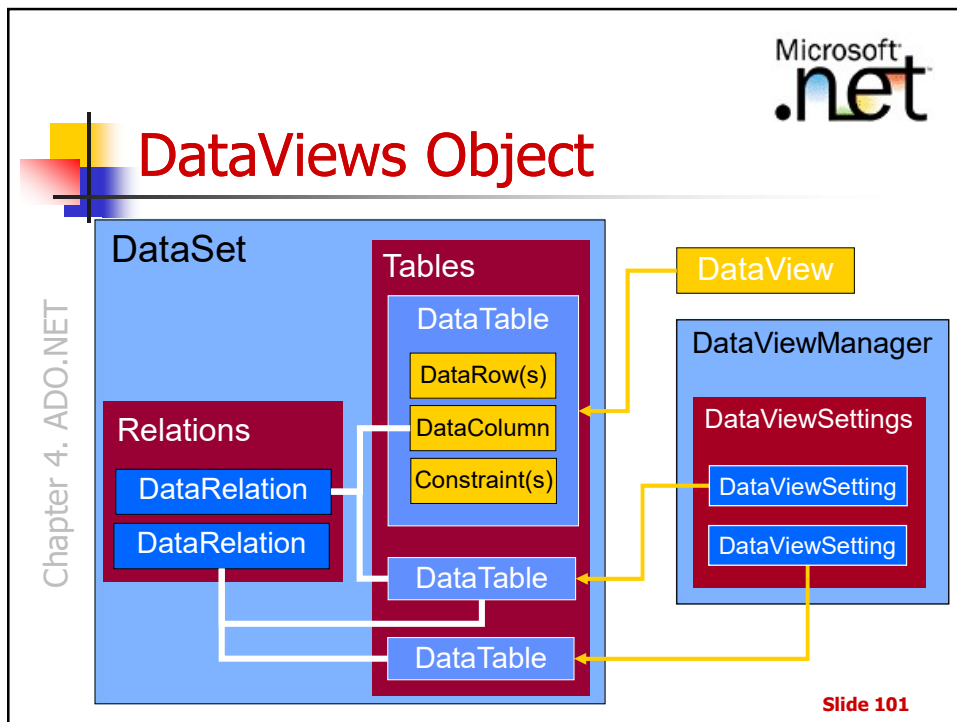
```
// Get the DataTable DataColumnns we want to relate...
DataColumn parentCol, childCol;
parentCol= DataSet.Tables["Customers"].Columns["CustID"];
childCol = DataSet.Tables["Orders"].Columns["CustID"];

// Create DataRelation with the name "CustomerOrders"...
DataRelation dr = new DataRelation("CustomerOrders",
                                   parentCol,
                                   childCol);

// Add the relation to the DataSet...
ds.Relations.Add(dr);
```

Slide 100

100



101

Chapter 4. ADO.NET

Microsoft .net

DataView Object

- Create multiple views on **DataTable** objects
- Bindable to user interface controls
- Properties of Interest:
 - **Table**: Retrieves or sets the associated **DataTable**
 - **Sort**: Gets or sets the table's sort columns and sort order
 - **RowFilter**: Gets or sets the expression used to filter rows
 - **RowStateFilter**: Gets or sets the row state filter
 - **None**, **Unchanged**, **New**, **Deleted**, **ModifiedCurrent**, and others

Slide 102

102

Microsoft
.net



DataView Object

Chapter 4. ADO.NET

```
// Code for myTable "Customers" with "Name" column not shown
DataView view1 = new DataView( myTable );
DataView view2 = new DataView( myTable );

// Creates Ascending view of Customers by "Name"
view1.Sort = "Name ASC";

// Set the view to show only modified (original) rows
view2.RowStateFilter= DataViewRowState.ModifiedOriginal;

// Bind to UI element(s)...
DataGrid myGrid = new DataGrid();
myGrid.SetDataBinding( view1, "Customer");

//...
```

Slide 103

103

Microsoft
.net



DataViewManager Object

Chapter 4. ADO.NET

- Similar to a **DataView** but **DataSet** oriented
- Used to create multiple views on a **DataSet**
 - Ability to automatically set filters on the tables
- Properties of Interest:
 - **DataViewSettings**: Gets the **DataView** for on each **DataTable**
 - **DataSet**: Gets or sets the **DataSet** to be viewed
- **CreateDataView** method
 - Creates a **DataView** on a **DataTable**

Slide 104

104



DataViewManager By Example

```
// Create the DataViewManager & views...
DataViewManager dvMgr = new DataViewManager( myDS );
dvMgr.CreateDataView( ds.Tables["Orders"] );
dvMgr.DataViewSettings["Orders"].Sort = "CustID ASC";

dvMgr.CreateDataView( ds.Tables["Customers"] );
dvMgr.DataViewSettings["Customers"].Sort = "Name DESC";

// Bind to a UI elements/controls...
dataGrid1.DataSource = viewMgr;
dataGrid1.DataMember = "Table1";

dataGrid2.DataSource = viewMgr;
dataGrid2.DataMember = "Table2";

// Update the control with the data...
dataGrid1.Update();
dataGrid2.Update();
```

Slide 105

105



Summary

- ADO.NET is the evolution of ADO
- It is a disconnected, Web-centric model
- Flexible in its ability to work with data
- Increases your ability to logically organize data
- Extensive support for XML
 - Facilitates working with and sharing data
- Interacts with a wide variety of data sources

Slide 106

106